Operating Systems
Sheet 1 Solutions

Name: Yehia Said Gewily
ID: 21010705
Date: February 2026

1 Basic Elements (1.1)

(a) Processor Register Categories

There are two main types of registers:

1. **User-visible registers:** These are the ones we can access directly in code (like general purpose registers, data registers, or the stack pointer).
2. **Control and status registers:** These are used by the CPU control unit and the OS to manage execution. Examples include the Program Counter (PC) and the Instruction Register (IR). User programs generally can't access these directly.

2 Evolution of the Microprocessor (1.2)

(a) GPUs for Streaming Signals

False. GPUs are optimized for parallel tasks like rendering graphics or running machine learning models. Streaming signals (like audio) are usually handled by Digital Signal Processors (DSPs).

(b) SoC Definition

True. A System on a Chip (SoC) puts everything needed for the system (CPU, Memory controller, DSP, I/O, radios) onto a single chip.

3 Instruction Execution (1.3)

(a) Four Distinct Actions

1. **Processor-Memory:** Moving data between the CPU and memory.
2. **Processor-I/O:** Moving data between the CPU and an external device.
3. **Data Processing:** Performing logic or arithmetic on data (e.g. ADD, SUB).
4. **Control:** Changing the execution sequence (e.g. JUMP or Branch instructions).

(b) Program Execution Trace

Opcodes used: `0011` = Load from I/O, `0101` = Add from Memory, `0111` = Store to I/O.
Initial State: PC = 300, Device 5 = 3, Memory[940] = 2.

Step 1: Fetch (PC = 300)	**Step 2: Execute**
Mem: 300 has `3005` **Registers:** PC → 301, IR → `3005`	**Decoding:** `3005` Opcode `0011` is "Load from I/O". Address is 5. **Action:** Load value 3 from Dev 5. **Result:** AC = 3.

Step 3: Fetch (PC = 301)	**Step 4: Execute**
Mem: 301 has `53AC` **Registers:** PC → 302, IR → `53AC`	**Decoding:** `53AC` Opcode `0101` is "Add from Mem". Target is Mem[940] (which is 2). **Action:** AC = AC + 2 **Result:** AC = 3 + 2 = 5.

Step 5: Fetch (PC = 302)	**Step 6: Execute**
Mem: 302 has `7006` **Registers:** PC → 303, IR → `7006`	**Decoding:** `7006` Opcode `0111` is "Store to I/O". Address is 6. **Action:** Send AC value to Dev 6. **Result:** Device 6 receives 5.

Final Output: Device 6 outputs **5**.

4 Interrupts (1.4)

(a) Program Interrupts

False. Program interrupts happen when the program does something illegal, like dividing by zero or trying to access protected memory. Normal completion is handled by I/O interrupts.

(b) Efficiency of Interrupts

If we didn't have interrupts, the CPU would have to constantly check (poll) the device to see if it's done. This wastes a huge amount of CPU time doing nothing.

With interrupts, the CPU sends the command and then goes off to do other work. When the device is ready, it "interrupts" the CPU. Even though there is a small overhead to save the current state and switch tasks, it's much more efficient than the CPU sitting idle waiting for a slow I/O device.

(c) Priority Scheme Example

Scenario: We have 3 devices with priorities P1 > P2 > P3.

- CPU is processing an interrupt for P3.
- P2 requests an interrupt. Since P2 > P3, the CPU pauses P3 and handles P2 immediately.
- While P2 is running, P1 requests an interrupt. Since P1 > P2, the CPU pauses P2 and runs P1.

Advantage: If we just disabled interrupts, high-priority tasks (like a network packet arriving or a sensor alert) would have to wait for low-priority tasks to finish. Priority schemes ensure urgent things get handled right away.

5 The Memory Hierarchy (1.5)

(a) Why the Hierarchy?

We need the hierarchy because there is no single memory that is fast, large, and cheap all at once.

- **Registers/Cache:** Fast but expensive and small.
- **Main Memory:** Slower but larger capacity.
- **Disk:** Very slow, but huge capacity and cheap.

By mixing these, we get a system that feels fast (because of cache) but has lots of storage (because of disk).

(b) Locality of Reference

This explains why caching works. Programs don't access memory randomly:

- **Temporal Locality:** If you use a variable now, you'll probably use it again soon (e.g., a loop counter).
- **Spatial Locality:** If you access address X, you'll probably access $X + 1$ soon (e.g., iterating through an array).

(c) Code Example

- **Temporal:** The variable `a[i]` is used inside the inner loop 20 times in a row. It stays in the cache/register.
- **Spatial:** The array is accessed sequentially (`a[0]`, then `a[1]`, etc.). Loading `a[0]` into the cache likely brings `a[1]` with it.

6 Cache Memory (1.6)

(a) Cost and Performance

1. Cost without cache:

$$\text{Cost} = S_m \times C_m = 256 \times 2^{20} \text{ bytes} \times 0.0001 \text{ cents/byte}$$
$$= 268,435,456 \times 0.0001 \approx \mathbf{26,843.55} \text{ cents} \approx \$\mathbf{268.44}$$

2. Cost with cache:

$$\begin{aligned}\text{Cost} &= (S_m \times C_m) + (S_c \times C_c)\\ &= 26,843.55 + (32,768 \times 0.1)\\ &= 26,843.55 + 3,276.8 = \mathbf{30,120.35} \text{ cents} \approx \$\mathbf{301.20}\end{aligned}$$

3. Performance Improvement (Hit ratio 0.85):

New Average Time:

$$T_{avg} = (0.85 \times 10) + (0.15 \times 100) = 8.5 + 15 = 23.5 \text{ ns}$$

Improvement over 100 ns:

$$\frac{100 - 23.5}{100} = \mathbf{76.5}\%$$

(b) Average Access Time (Read/Write Mix)

- **Reads (80%):** $T_r = 0.9(60) + 0.1(300) = 54 + 30 = 84$ ns.
- **Writes (20%):** Write-through means we always go to main memory, so $T_w = 300$ ns.

$$T_{avg} = (0.8 \times 84) + (0.2 \times 300) = 67.2 + 60 = \mathbf{127.2} \text{ ns}$$

7 Direct Memory Access (1.7)

(a) Transfer Rates

1. Programmed I/O:

The CPU is 95% busy, so only 5% is free for I/O.

$$\text{Free cycles} = 1,000,000 \times 0.05 = 50,000 \text{ instructions/s}$$

It takes 2 instructions per word:

$$\text{Rate} = \frac{50,000}{2} = \mathbf{25,000} \text{ words/s}$$

2. DMA:

Total Bus Cycles $= 10^6 \times 5 = 5,000,000$ cycles/s. Background tasks use: $950,000$ instr $\times$ 3 cycles $= 2,850,000$ cycles. Remaining cycles for DMA:

$$5,000,000 - 2,850,000 = \mathbf{2,150,000} \text{ words/s}$$

(DMA uses 1 cycle per word).

8 Multiprocessor (1.8)

(a) SMP Advantages

- **Performance:** True parallelism allows multiple processes to run at the exact same time.
- **Reliability:** If one CPU dies, the others can keep the system running (graceful degradation).
- **Scaling:** It's easier to just add another CPU to the bus than to replace the whole system.

(b) Multicore

Putting multiple cores on one chip (cmp) is better because:

1. We hit the "Power Wall" – making single cores faster made them too hot.
2. Communication between cores on the same chip is much faster than between separate chips.